ClassSync Pro

Class Timetable & Clash Detection System

Software Requirements Specification

Version 1.0

Technology Stack: Python | Flask | SQLAlchemy | SQLite / PostgreSQL

May 2026

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience
- 1.4 Product Scope

2. Overall Description
- 2.1 Product Perspective
- 2.2 User Roles and Permissions
- 2.3 Database Schema & Table Design
- 2.4 Operating Environment
- 2.5 Assumptions and Dependencies

3. Functional Requirements – Full Function List
- 3.1 Authentication & Session Management
- 3.2 User Management (CRUD)
- 3.3 Teacher Management (CRUD)
- 3.4 Course Management (CRUD)
- 3.5 Room Management (CRUD)
- 3.6 Time Slot Management (CRUD)
- 3.7 Section / Student Group Management (CRUD)
- 3.8 Class Schedule Management – Main Table (CRUD)
- 3.9 Clash Detection Engine
- 3.10 Dashboard & Reporting
- 3.11 Import / Export
- 3.12 Role-Based Access Control

4. Use Cases

5. Diagrams
- 5.1 DFD Level 0 – Context Diagram
- 5.2 DFD Level 1 – Process Decomposition
- 5.3 Activity Diagram – Admin Workflow
- 5.4 Activity Diagram – Teacher Workflow
- 5.5 Class Diagram
- 5.6 Entity-Relationship (ER) Diagram

6. External Interface Requirements

7. Non-Functional Requirements

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and detailed description of the ClassSync Pro – Class Timetable and Clash Detection System. The system is a role-based web application built with Python (Flask framework) and designed to automate the scheduling of academic classes while automatically detecting and resolving entity conflicts such as teacher double-booking, room double-allocation, and section overlaps.

This document serves as the authoritative technical blueprint for developers, testers, system architects, and academic stakeholders. It defines all functional requirements, system behaviors, data structures, diagrams, use cases, and non-functional properties of the system.

1.2 Document Conventions

This document follows IEEE 830-1998 SRS standards. Heading levels indicate the hierarchy of information. Bold text denotes table names, role names, and important system entities. Italic text marks technical terms such as clash, time_slot, or section when first introduced. All module identifiers use lowercase_snake_case consistent with the Python/Flask codebase. Requirement identifiers follow the pattern FR-XX (Functional Requirement) and NFR-XX (Non-Functional Requirement).

1.3 Intended Audience

This document is intended for the following stakeholders:

- Backend Developers – to implement Flask routes, SQLAlchemy models, and clash detection logic.
- Frontend Developers – to design role-aware Jinja2 templates and dashboards.
- Database Architects – to implement the relational schema and indexing strategy.
- System Testers / QA – to write and execute test cases against stated requirements.
- Project Supervisors / Instructors – to evaluate scope, completeness, and correctness.
- University Administrators – to understand system capabilities before deployment.

1.4 Product Scope

ClassSync Pro is a centralized, browser-based timetable management platform for academic institutions. It allows administrators to construct, maintain, and validate semester-wide class schedules across multiple sections, rooms, time slots, and teachers. The system enforces four hierarchical user roles – Super Admin, Admin, Manager, and Teacher – each with distinct access rights. A built-in Clash Detection Engine automatically identifies and reports scheduling conflicts in real time. Data can be exported to CSV/Excel and imported in bulk, enabling rapid semester setup.

2. Overall Description

2.1 Product Perspective

ClassSync Pro is a standalone web application that replaces manual spreadsheet-based timetabling. It integrates a relational database (SQLite for development; PostgreSQL for production) accessed through SQLAlchemy ORM, a Flask-powered REST-like backend, and server-side rendered Jinja2 HTML templates with Bootstrap 5 for a responsive UI. The system does not depend on any third-party timetable or ERP system, though a REST API layer can be added in future versions for LMS integration.

2.2 User Roles and Permissions

The system enforces four hierarchical roles. Each higher role inherits all permissions of lower roles. No self-registration is available; accounts are created only by a higher-privileged user.

Role	Created By	Can Create Roles	Schedule CRUD	Clash View	Import/Export	System Config
Super Admin	System (seeded)	Admin, Manager, Teacher	Full	Yes	Yes	Yes
Admin	Super Admin	Manager, Teacher	Full	Yes	Yes	No
Manager	Admin or above	Teacher only	Read + Write	Yes	Export only	No
Teacher	Manager or above	None	Read own schedule	Own clashes	No	No

2.3 Database Schema & Table Design

The system comprises eight core relational tables. The relationships between them cover all three cardinality types: one-to-one, one-to-many, and many-to-many.

Table: users

Stores all system accounts regardless of role.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
username	VARCHAR(50) UNIQUE	Login identifier
email	VARCHAR(100) UNIQUE	Contact email
password_hash	VARCHAR(255)	Bcrypt hashed password
role	ENUM(super_admin,admin,manager,teacher)	Access level
is_active	BOOLEAN	Soft-enable/disable flag
created_by	INTEGER FK → users.id	Who created this account
created_at	DATETIME	Timestamp of creation

Table: teachers

Extended profile for users with role='teacher'. One-to-one with users.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
user_id	INTEGER FK → users.id UNIQUE	One-to-one link to users
employee_code	VARCHAR(20) UNIQUE	Official staff ID
department	VARCHAR(100)	Academic department
designation	VARCHAR(100)	e.g., Lecturer, Professor
phone	VARCHAR(20)	Contact number
max_hours_per_week	INTEGER DEFAULT 20	Weekly teaching cap

Table: courses

Academic courses/subjects that can be scheduled.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
code	VARCHAR(20) UNIQUE	e.g., CS-301
title	VARCHAR(150)	Full course name
credit_hours	INTEGER	Academic credit weight
course_type	ENUM(theory,lab,tutorial)	Delivery type
department	VARCHAR(100)	Owning department
description	TEXT	Optional course outline
is_active	BOOLEAN DEFAULT TRUE	Soft delete flag

Table: rooms

Physical or virtual locations where classes are held.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
room_number	VARCHAR(20) UNIQUE	e.g., LH-3, CS-Lab-1
building	VARCHAR(100)	Building name/block
capacity	INTEGER	Maximum seat count
room_type	ENUM(lecture_hall,lab,seminar_room,auditorium)	Room category

has_projector	BOOLEAN DEFAULT FALSE	AV equipment flag
has_ac	BOOLEAN DEFAULT FALSE	Comfort flag
is_available	BOOLEAN DEFAULT TRUE	Enabled/disabled

Table: time_slots

Predefined time blocks that can be assigned to schedule entries.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
day_of_week	ENUM(Monday … Friday,Saturday)	Day label
start_time	TIME	e.g., 08:00
end_time	TIME	e.g., 09:30
slot_label	VARCHAR(30)	e.g., 1st Period
is_break	BOOLEAN DEFAULT FALSE	Marks prayer/lunch breaks

Table: sections

Student groups (classes) that follow a shared timetable.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
name	VARCHAR(50) UNIQUE	e.g., BSCS-3A
semester	INTEGER	1–8 or 1–4
academic_year	VARCHAR(9)	e.g., 2025-2026
program	VARCHAR(100)	e.g., BSCS, BBA
student_count	INTEGER DEFAULT 0	Enrolled students
is_active	BOOLEAN DEFAULT TRUE	Soft delete flag

Table: course_teacher (many-to-many)

Assigns which teachers are qualified to teach which courses. Many-to-many between teachers and courses.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
teacher_id	INTEGER FK → teachers.id	Teacher reference

course_id	INTEGER FK → courses.id	Course reference
is_primary	BOOLEAN DEFAULT TRUE	Primary vs guest teacher

Table: class_time_schedule (main table)

The central scheduling table. Each row is one class session per week. Clash detection operates on this table.

Column Name	Data Type / Constraint	Description
id	INTEGER PK	Auto-increment primary key
section_id	INTEGER FK → sections.id	Student group
course_id	INTEGER FK → courses.id	Subject being taught
teacher_id	INTEGER FK → teachers.id	Assigned teacher
room_id	INTEGER FK → rooms.id	Physical location
time_slot_id	INTEGER FK → time_slots.id	Day + time block
semester	INTEGER	Semester number
academic_year	VARCHAR(9)	Session year
session_type	ENUM(regular,makeup,extra)	Entry type
created_by	INTEGER FK → users.id	Who entered record
created_at	DATETIME	Record timestamp
is_active	BOOLEAN DEFAULT TRUE	Soft delete flag

2.4 Operating Environment

- Server: Python 3.10+, Flask 3.x, SQLAlchemy 2.x, Alembic (migrations)
- Database: SQLite (development) / PostgreSQL 15 (production)
- Frontend: Jinja2 templates, Bootstrap 5, jQuery 3.x, Chart.js
- Authentication: Flask-Login, Flask-Bcrypt
- Export: pandas, openpyxl; Import: pandas CSV/Excel parser
- Deployment: Linux/Ubuntu server, Nginx + Gunicorn, or Docker container
- Browser Support: Chrome 110+, Firefox 110+, Edge 110+, Safari 16+

2.5 Assumptions and Dependencies

- All users access the system through a web browser; no native mobile app is required in v1.0.
- Users have basic familiarity with academic scheduling concepts (semester, section, credit hours).
- Time slots are predefined by Super Admin during initial system setup.

- A teacher's schedule is only editable by Manager-level or above.
- Clash detection runs synchronously at the time of schedule entry and does not prevent save – it flags and warns.
- Import files must be CSV or .xlsx with a predefined column header template provided by the system.

3. Functional Requirements – Complete Function List

3.1 Authentication & Session Management

Function ID	Function Name	Description & Behavior
F-AUTH-01	**Login**	Validates username/password, creates secure session. Redirects to role-specific dashboard on success. Displays error on failure. No brute-force after 5 attempts (account lock 15 min).
F-AUTH-02	**Logout**	Destroys session, clears all cookies, redirects to login page.
F-AUTH-03	**Forgot Password**	Generates a time-limited (1 hr) reset token sent to the user's registered email via Flask-Mail.
F-AUTH-04	**Reset Password**	Validates token, allows secure new password entry with bcrypt re-hash.
F-AUTH-05	**Session Timeout**	Auto-logout after 30 minutes of inactivity. Configurable by Super Admin.
F-AUTH-06	**Change Own Password**	Any logged-in user can change their own password after confirming the current one.

3.2 User Management (CRUD)

Function ID	Function Name	Description & Behavior
F-USR-01	**Create User**	Super Admin / Admin / Manager create user accounts with role ≤ their own. Auto-generated temporary password is shown once.
F-USR-02	**List Users**	Paginated, searchable, filterable (by role, department, active status) user table.
F-USR-03	**View User Detail**	Shows full profile, role, creation date, and activity log entries.
F-USR-04	**Edit User**	Update username, email, role, department. Cannot elevate role above own.
F-USR-05	**Activate / Deactivate User**	Soft-delete: deactivated users cannot log in. Schedule records are retained.
F-USR-06	**Reset User Password**	Admin or above can force-reset a lower-level user's password.
F-USR-07	**Delete User**	Hard delete only if user has no associated schedule records.

3.3 Teacher Management (CRUD)

Function ID	Function Name	Description & Behavior
F-TCH-01	Create Teacher Profile	Manager or above creates teacher profile linked to a user account (user_id FK). Captures department, designation, max weekly hours.
F-TCH-02	List Teachers	Table with search/filter by department, designation, active status. Shows weekly load summary.
F-TCH-03	View Teacher	Full profile including assigned courses, weekly schedule view, and current clash alerts.
F-TCH-04	Edit Teacher	Update all teacher profile fields except user_id.
F-TCH-05	Deactivate Teacher	Soft-delete; schedule entries remain but future assignments blocked.
F-TCH-06	Assign Courses to Teacher	Many-to-many assignment via course_teacher junction table. Mark as primary or guest.
F-TCH-07	View Teacher Schedule	Calendar/grid view of a single teacher's entire weekly timetable.
F-TCH-08	Teacher Load Report	Shows how many hours/week a teacher is scheduled vs their maximum cap.

3.4 Course Management (CRUD)

Function ID	Function Name	Description & Behavior
F-CRS-01	Create Course	Admin or above creates course with code, title, credit hours, type (theory/lab/tutorial), and department.
F-CRS-02	List Courses	Searchable/filterable table by department, type, active status.
F-CRS-03	Edit Course	Update any field; code must remain unique.
F-CRS-04	Deactivate Course	Soft-delete; existing schedule entries not removed.
F-CRS-05	View Course Teachers	Lists all teachers assigned to the course via course_teacher.

3.5 Room Management (CRUD)

Function ID	Function Name	Description & Behavior
F-RM-01	Create Room	Admin or above adds room with number, building, capacity, type, and facilities.
F-RM-02	List Rooms	Table with filter by type, capacity range, availability.
F-RM-03	Edit Room	Update any room attribute.

F-RM-04	**Toggle Availability**	Mark a room as unavailable (e.g., under renovation) without deleting.
F-RM-05	**View Room Schedule**	Grid showing all schedule entries assigned to a specific room across all days.

3.6 Time Slot Management (CRUD)

Function ID	Function Name	Description & Behavior
F-TS-01	**Create Time Slot**	Super Admin defines day + start/end time + label. Validates no overlap with existing slots on same day.
F-TS-02	**List Time Slots**	Weekly grid view and tabular list with day filter.
F-TS-03	**Edit Time Slot**	Update times and label; system warns if existing schedule entries use this slot.
F-TS-04	**Delete Time Slot**	Only allowed if no schedule entry references it.
F-TS-05	**Mark Break**	Flag slot as break period (lunch, prayer) – cannot be scheduled.

3.7 Section / Student Group Management (CRUD)

Function ID	Function Name	Description & Behavior
F-SEC-01	**Create Section**	Manager or above creates student group with name, program, semester, academic year, student count.
F-SEC-02	**List Sections**	Filterable by program, semester, academic year, active status.
F-SEC-03	**Edit Section**	Update any field; name must remain unique.
F-SEC-04	**Deactivate Section**	Soft-delete preserving historical schedule.
F-SEC-05	**View Section Timetable**	Full weekly timetable in grid format for a selected section.

3.8 Class Schedule Management – Main Table (CRUD)

The class_time_schedule table is the heart of the system. Every entry represents one recurring weekly class session.

Function ID	Function Name	Description & Behavior
F-SCH-01	**Create Schedule Entry**	Manager or above selects section + course + teacher + room + time_slot + semester/year. System immediately runs clash check and

		displays any conflicts before save. Entry is saved regardless (soft warning), but flagged if clashes exist.
F-SCH-02	**List Schedule Entries**	Master schedule view with multi-filter: section, teacher, room, day, course, semester, year. Supports timetable grid and list views.
F-SCH-03	**Edit Schedule Entry**	Modify any field. Re-runs clash detection on save.
F-SCH-04	**Delete Schedule Entry**	Soft delete (is_active=FALSE) with reason capture.
F-SCH-05	**Bulk Schedule Entry**	Upload CSV/Excel with multiple entries; each row validated and clashes pre-reported before commit.
F-SCH-06	**Copy Timetable**	Duplicate a section's full schedule to a new semester/year with one click.
F-SCH-07	**Publish Timetable**	Mark a section's timetable as 'published' making it visible to Teacher role.
F-SCH-08	**Timetable Grid View**	Visual 5- or 6-day week × time-slot matrix for any section or teacher.

3.9 Clash Detection Engine

The Clash Detection Engine (CDE) is a core subsystem that queries class_time_schedule for conflicts at the same time_slot_id on the same day. It detects three clash types:

Function ID	Function Name	Description & Behavior
F-CDE-01	**Teacher Clash**	Detects if the same teacher_id is assigned to two or more entries with the same time_slot_id (same day+time). Result: highlighted in red on schedule grid with teacher name and conflicting sections listed.
F-CDE-02	**Room Clash**	Detects if the same room_id is double-booked at the same time_slot_id. Result: amber warning showing which sections/courses share the room.
F-CDE-03	**Section Clash**	Detects if the same section_id has two courses at the same time_slot_id (impossible by design – triggers hard validation error).
F-CDE-04	**Real-Time Clash Check**	Runs before each create/edit save. Returns a structured JSON clash report to the UI via an AJAX call (/api/check-clash). No page reload required.
F-CDE-05	**Bulk Clash Report**	On-demand scan of an entire semester's schedule. Generates a paginated clash report listing every conflict with resolution suggestions.
F-CDE-06	**Clash Resolution Suggestions**	For teacher clashes, suggests alternative free time slots for that teacher. For room clashes, lists available rooms of the same type at that time.

F-CDE-07	**Clash Dashboard Widget**	Super Admin / Admin dashboard shows count of active clashes by type with drill-down links.
F-CDE-08	**Export Clash Report**	Download the full clash report as CSV or PDF.

3.10 Dashboard & Reporting

Function ID	Function Name	Description & Behavior
F-DSH-01	**Super Admin Dashboard**	System stats: total users, teachers, sections, rooms, courses, schedule entries; clash count by type; recent activity log; quick-action buttons.
F-DSH-02	**Admin Dashboard**	Schedule overview for current semester; clash summary; pending approvals (if workflow enabled); quick links to CRUD modules.
F-DSH-03	**Manager Dashboard**	Sections they manage; clash alerts; teacher load chart; quick add schedule entry.
F-DSH-04	**Teacher Dashboard**	Own weekly timetable (read-only grid); total assigned hours; upcoming classes; clash alerts on own schedule.
F-DSH-05	**Activity Log**	Admin+ can view timestamped log of all CREATE/UPDATE/DELETE actions with user, table, record ID, and before/after values.
F-DSH-06	**Teacher Load Report**	Bar chart of hours/week per teacher vs their max_hours_per_week.
F-DSH-07	**Room Utilization Report**	Heatmap grid showing room occupancy percentage by day and time slot.
F-DSH-08	**Section Timetable Report**	Printable PDF timetable for each section.

3.11 Import / Export

Function ID	Function Name	Description & Behavior
F-IE-01	**Export Schedule (CSV)**	Export full semester timetable or filtered subset as CSV. Available to Admin+.
F-IE-02	**Export Schedule (Excel)**	Same as above but formatted .xlsx with color-coded clash rows.
F-IE-03	**Export Schedule (PDF)**	Printable section timetable with institution branding. Available to Manager+.

F-IE-04	**Import Schedule (CSV/Excel)**	Admin+ uploads a template-format file; system validates all foreign keys and runs clash detection; displays preview with error rows highlighted before final commit.
F-IE-05	**Download Import Template**	Pre-formatted CSV/Excel with correct headers and sample data for each importable table.
F-IE-06	**Export Teachers / Courses / Rooms**	Individual entity exports available to Admin+.
F-IE-07	**Import Teachers / Courses**	Bulk create from file with validation and duplicate detection.

3.12 Role-Based Access Control (RBAC)

Every Flask route and template block is protected by a role decorator. The table below summarises module access by role.

Module / Page	Super Admin	Admin	Manager	Teacher
User Management	Full CRUD	Create/Edit/Deactivate (≤Admin)	Create Teacher only	View own profile
Teacher Profiles	Full CRUD	Full CRUD	Full CRUD	View own
Courses	Full CRUD	Full CRUD	Read + Assign	Read only
Rooms	Full CRUD	Full CRUD	Read only	Read only
Time Slots	Full CRUD	Read only	Read only	Read only
Sections	Full CRUD	Full CRUD	Full CRUD	Read only
Schedule (create)	Yes	Yes	Yes	No
Schedule (read)	All	All	All	Own entries only
Clash Detection	All clashes	All clashes	All clashes	Own clashes
Import	Yes	Yes	No	No
Export	Yes	Yes	Yes (PDF/CSV)	No
Activity Log	Yes	Yes	No	No
System Settings	Yes	No	No	No

4. Use Cases

The following use cases represent the primary interactions between actors and the ClassSync Pro system. Each use case is linked to functional requirements identified in Section 3.

ID	Use Case	Actor	Main Goal	Pre-condition	FR Refs
UC-01	Super Admin Configures the System	Super Admin	Super Admin navigates to System Settings.	Super Admin is logged in.	F-AUTH-05, F-TS-01, F-USR-01, F-DSH-01
UC-02	Admin Creates and Assigns Entities	Admin	Admin creates rooms, courses, and sections.	Admin is logged in. Time slots exist.	F-RM-01, F-CRS-01, F-SEC-01, F-TCH-01, F-TCH-06, F-USR-01
UC-03	Manager Builds a Section Timetable	Manager	Manager opens Schedule → Create Entry.	Manager is logged in. Sections, courses, teachers, rooms, and time slots exist.	F-SCH-01 to F-SCH-07, F-CDE-01 to F-CDE-04
UC-04	Manager Detects and Resolves Clashes	Manager	Manager opens Clash Report (F-CDE-05).	A semester schedule exists with at least one clash.	F-CDE-01 to F-CDE-08, F-SCH-03
UC-05	Teacher Views Own Timetable	Teacher	Teacher lands on their dashboard (F-DSH-04).	Teacher is logged in. Their timetable is published.	F-DSH-04, F-TCH-07
UC-06	Admin Imports Bulk Schedule	Admin	Admin downloads the import template (F-IE-05).	Admin has a filled import template file.	F-IE-04, F-IE-05, F-CDE-05
UC-07	Admin Exports and Prints Timetable	Admin / Manager	User selects section and semester.	A published timetable exists.	F-IE-03, F-SCH-08

UC-01: Super Admin Configures the System

Actor: Super Admin | Pre-condition: Super Admin is logged in.

Main Flow:

1. Super Admin navigates to System Settings.
2. Defines session timeout, institution name, and semester dates.
3. Creates initial time slots for the week.
4. Creates Admin accounts for department heads.
5. System confirms all settings saved.

Post-condition: System is ready for scheduling.

Related Functions: F-AUTH-05, F-TS-01, F-USR-01, F-DSH-01

UC-02: Admin Creates and Assigns Entities

Actor: Admin | Pre-condition: Admin is logged in. Time slots exist.

Main Flow:

6. Admin creates rooms, courses, and sections.
7. Admin creates teacher user accounts.
8. Admin assigns courses to teachers via course_teacher.
9. Admin delegates schedule entry to a Manager.

Post-condition: Prerequisite data for scheduling is in place.

Related Functions: F-RM-01, F-CRS-01, F-SEC-01, F-TCH-01, F-TCH-06, F-USR-01

UC-03: Manager Builds a Section Timetable

Actor: Manager | Pre-condition: Manager is logged in. Sections, courses, teachers, rooms, and time slots exist.

Main Flow:

10. Manager opens Schedule → Create Entry.
11. Selects section, course, teacher, room, and time slot.
12. System runs real-time clash check (F-CDE-04) and shows result.
13. If no clash: Manager saves entry.
14. If clash detected: Manager either modifies selections or saves with clash warning.
15. Manager repeats for all classes in the section.
16. Manager publishes timetable (F-SCH-07).

Post-condition: Section timetable is complete and visible to teachers.

Related Functions: F-SCH-01 to F-SCH-07, F-CDE-01 to F-CDE-04

UC-04: Manager Detects and Resolves Clashes

Actor: Manager | Pre-condition: A semester schedule exists with at least one clash.

Main Flow:

17. Manager opens Clash Report (F-CDE-05).
18. Views list of all clashes with type (teacher/room/section).
19. For teacher clash: system suggests free slots (F-CDE-06).
20. Manager edits the offending schedule entry to a new time/room.
21. System re-validates and confirms clash resolved.
22. Manager exports resolved report (F-CDE-08).

Post-condition: All clashes in the selected semester are resolved.

Related Functions: F-CDE-01 to F-CDE-08, F-SCH-03

UC-05: Teacher Views Own Timetable

Actor: Teacher | Pre-condition: Teacher is logged in. Their timetable is published.

Main Flow:

23. Teacher lands on their dashboard (F-DSH-04).
24. Sees weekly timetable grid with course, section, room, time.
25. Teacher clicks a slot to see full details.
26. Teacher views any clash alerts on their own entries.

Post-condition: Teacher is informed of their schedule.

Related Functions: F-DSH-04, F-TCH-07

UC-06: Admin Imports Bulk Schedule

Actor: Admin | Pre-condition: Admin has a filled import template file.

Main Flow:

27. Admin downloads the import template (F-IE-05).
28. Fills data in CSV/Excel and uploads via Import Schedule (F-IE-04).
29. System validates all FK references and runs clash detection per row.
30. Admin sees preview table with valid rows (green) and error rows (red).
31. Admin corrects errors or proceeds to commit valid rows.
32. System inserts valid entries and reports final import summary.

Post-condition: Multiple schedule entries are created efficiently.

Related Functions: F-IE-04, F-IE-05, F-CDE-05

UC-07: Admin Exports and Prints Timetable

Actor: Admin / Manager | Pre-condition: A published timetable exists.

Main Flow:

33. User selects section and semester.
34. Clicks Export → PDF (F-IE-03).
35. System generates printable PDF with section timetable grid.
36. User downloads and prints for notice boards.

Post-condition: Physical timetable is available for distribution.

Related Functions: F-IE-03, F-SCH-08

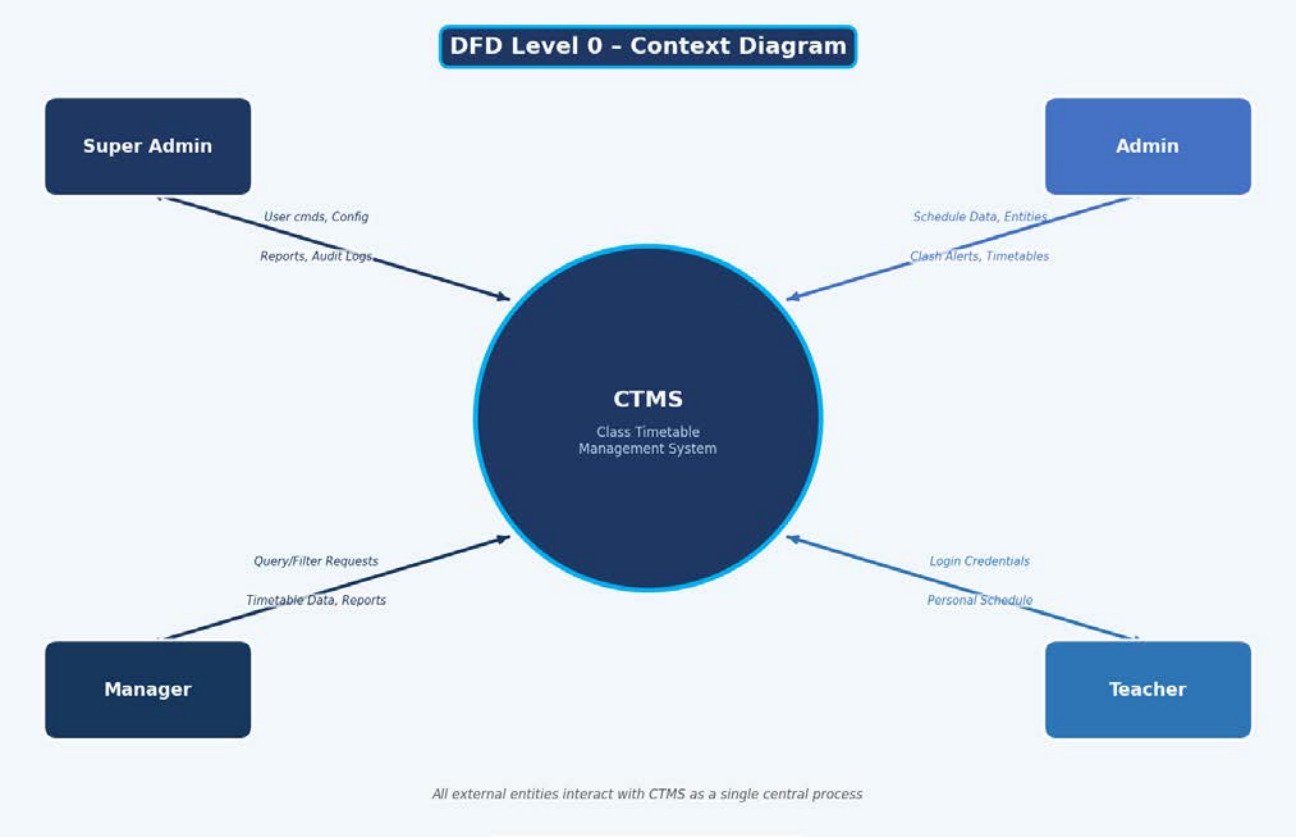
5. Diagrams

This section provides textual specifications for all required diagrams. Each diagram should be drawn using a UML or DFD tool such as draw.io, Lucidchart, or StarUML and inserted here. The descriptions below serve as the complete specification for each diagram.

5.1 Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD shows the ClassSync Pro system as a single process interacting with all external entities. It provides a bird's-eye view of data flowing into and out of the system boundary.

External Entity	Data Flow Direction	Data Description
Super Admin	→ System	System settings, time slot definitions, admin account creation
Admin	→ System	Entity data (teachers, rooms, courses, sections), schedule entries
Admin	← System	Reports, clash summaries, activity logs, export files
Manager	→ System	Schedule entries, bulk import files, section assignments
Manager	← System	Timetable grids, clash reports, export PDFs
Teacher	→ System	Login credentials, profile update requests
Teacher	← System	Personal timetable view, clash alerts, load report
Database (SQLite/PostgreSQL)	↔ System	All persistent entity and schedule data
File System	← System	Generated export files (CSV, Excel, PDF)
Email Server	← System	Password reset tokens



5.2 Data Flow Diagram – Level 1 (Process Decomposition)

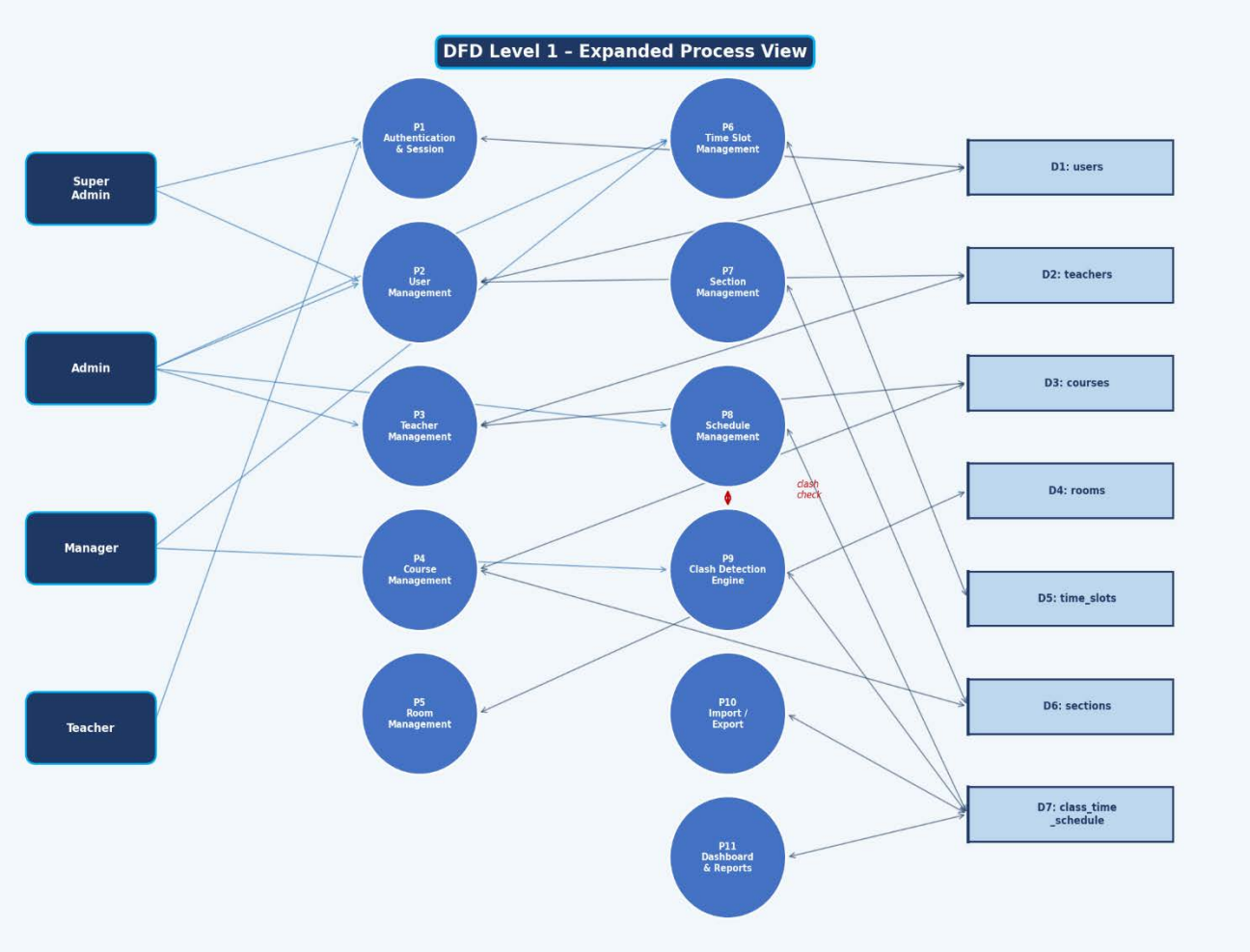
The Level 1 DFD decomposes the central ClassSync Pro process into its major sub-processes. The following processes are identified:

P#	Process Name	Description, Inputs, Outputs
P1	**Authentication & Authorization**	Receives login credentials → validates against users table → returns role-specific session token. Inputs: Username, Password. Outputs: Session, Role, Error Message.
P2	**User & Role Management**	Creates, updates, deactivates users. Enforces hierarchy (cannot create role above own). Inputs: User form data. Outputs: Updated users table, Audit log entry.
P3	**Entity Configuration**	Manages teachers, courses, rooms, time slots, sections. Each sub-process reads/writes its respective table. Inputs: Entity form data. Outputs: Updated tables, FK validation errors.
P4	**Schedule Entry Management**	Core CRUD for class_time_schedule. Reads all entity tables to populate dropdowns. Writes schedule entries. Inputs: Schedule form data. Outputs: Schedule record, Clash check trigger.
P5	**Clash Detection Engine**	Reads class_time_schedule filtered by time_slot_id. Identifies teacher, room, section clashes. Inputs: Candidate schedule entry. Outputs: Clash report JSON, Resolution suggestions.
P6	**Dashboard & Reporting**	Aggregates data from all tables. Generates role-specific views and reports. Inputs: Role, Date filters. Outputs: Dashboard data, Report files.

P7	**Import / Export**	Parses upload files, validates, maps to DB tables. Generates download files from DB queries. Inputs: CSV/Excel file. Outputs: Import summary, Download files.

Data Stores connected to Level 1 processes:

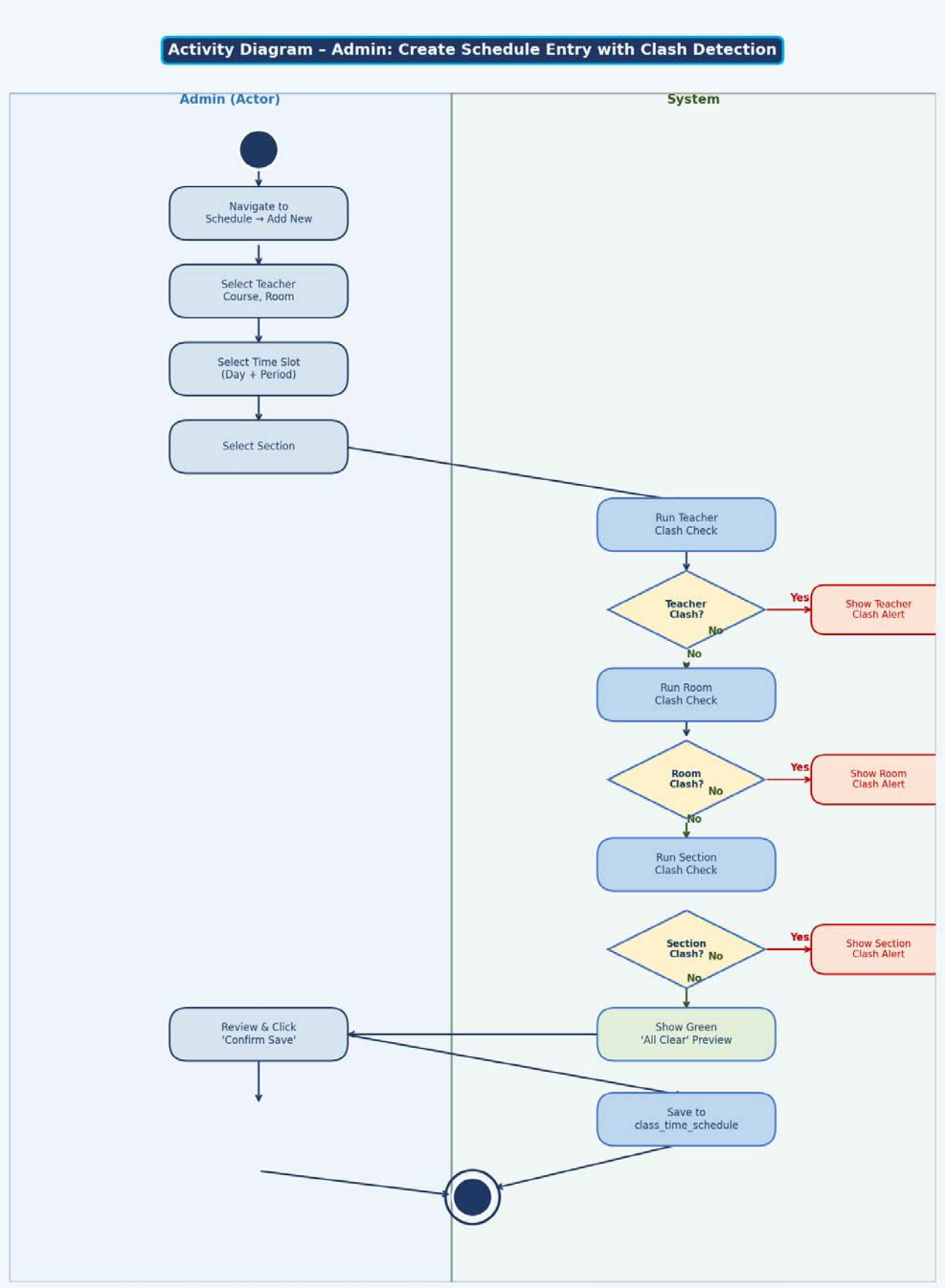
- DS1: users – read by P1, P2; written by P2
- DS2: teachers – read/written by P3; read by P4, P6
- DS3: courses – read/written by P3; read by P4, P6
- DS4: rooms – read/written by P3; read by P4, P5, P6
- DS5: time_slots – read/written by P3; read by P4, P5
- DS6: sections – read/written by P3; read by P4, P5, P6
- DS7: course_teacher – read/written by P3; read by P4
- DS8: class_time_schedule – read/written by P4; read by P5, P6, P7



5.3 Activity Diagram – Admin Workflow

The following activity sequence describes the Admin's primary workflow for setting up and managing a new semester schedule:

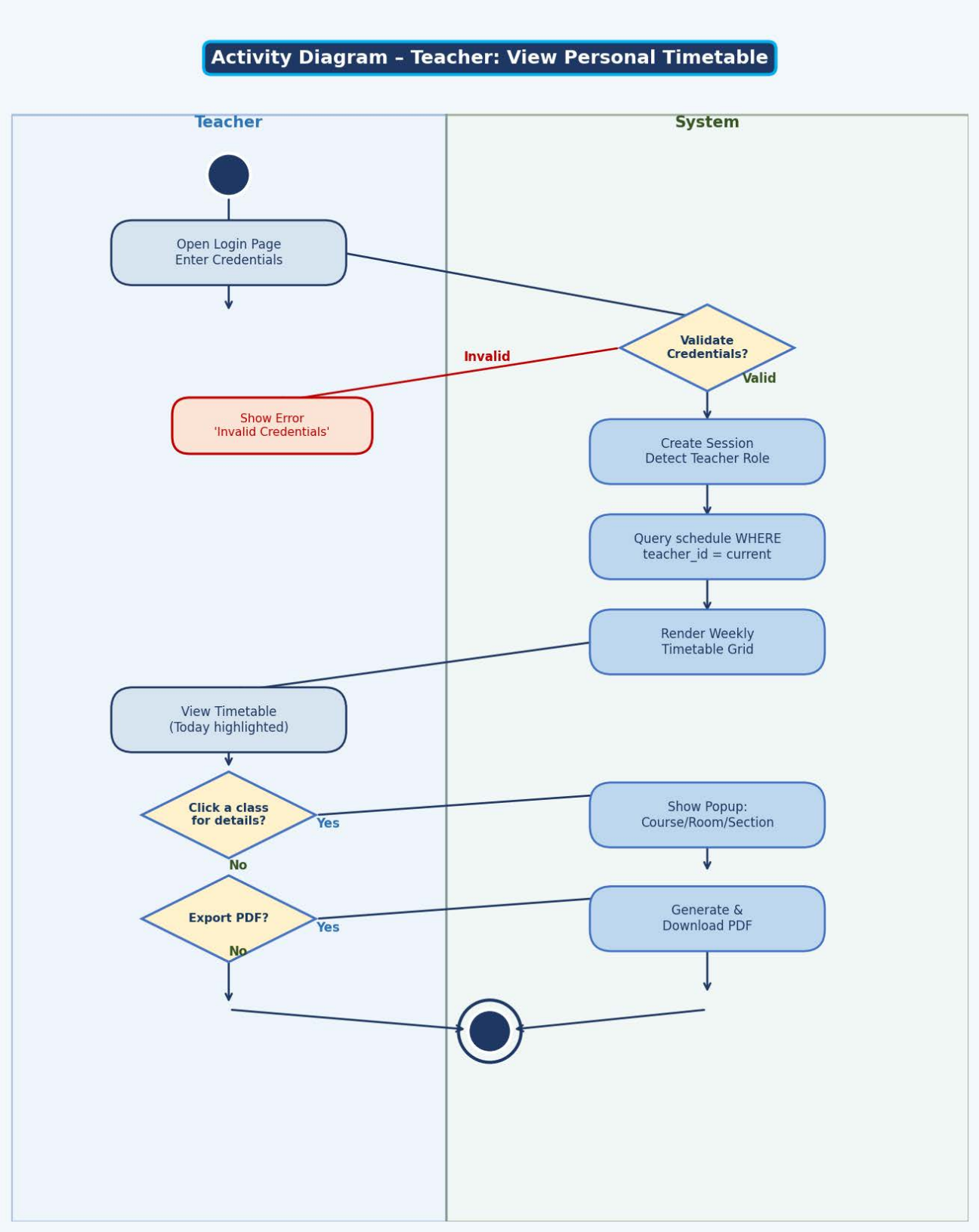
37. **START: Admin logs in → system validates credentials → redirects to Admin Dashboard.**
38. Admin creates/verifies rooms are configured (F-RM-01).
39. Admin creates/verifies courses exist (F-CRS-01).
40. Admin creates teacher user accounts and links teacher profiles (F-USR-01, F-TCH-01).
41. Admin assigns courses to teachers via course_teacher (F-TCH-06).
42. Admin creates sections/student groups for the semester (F-SEC-01).
43. Admin (or delegates to Manager) creates class schedule entries one-by-one or via bulk import (F-SCH-01 / F-IE-04).
44. DECISION: Clashes detected? → YES: View clash report, edit offending entries, re-validate. → NO: Continue.
45. Admin publishes all section timetables (F-SCH-07).
46. Admin exports PDF timetables for notice boards (F-IE-03).
47. DECISION: More clashes reported by teachers? → YES: Go back to step 8. → NO: END.
48. **END: Semester timetable is live.**


Activity Diagram – Admin: Create Schedule Entry with Clash Detection
Admin (Actor)
System
Navigate to Schedule → Add New
Select Teacher Course, Room
Select Time Slot (Day + Period)
Select Section
Run Teacher Clash Check
Teacher Clash?
Yes
Show Teacher Clash Alert
No
Run Room Clash Check
Room Clash?
Yes
Show Room Clash Alert
No
Run Section Clash Check
Section Clash?
Yes
Show Section Clash Alert
No
Show Green 'All Clear' Preview
Review & Click 'Confirm Save'
Save to class_time_schedule

5.4 Activity Diagram – Teacher Workflow

The Teacher's interaction with the system is intentionally read-only, ensuring data integrity:

49. **START: Teacher receives login credentials from Manager.**
50. Teacher logs in → system validates → redirects to Teacher Dashboard (F-DSH-04).
51. DECISION: Timetable published for this semester? → NO: Dashboard shows 'No published timetable yet'. → YES: Continue.
52. Teacher views weekly timetable grid – courses, rooms, time slots displayed.
53. Teacher checks clash alerts section – highlights any clashes on their own entries.
54. Teacher clicks a schedule slot to view course details, section name, room number.
55. DECISION: Teacher finds an error (wrong room/time)? → YES: Teacher contacts Manager to correct (system has no edit access for Teacher role). → NO: Continue.
56. Teacher views their load report – hours/week vs max cap (F-TCH-08).
57. **END: Teacher is informed and prepared for the semester.**

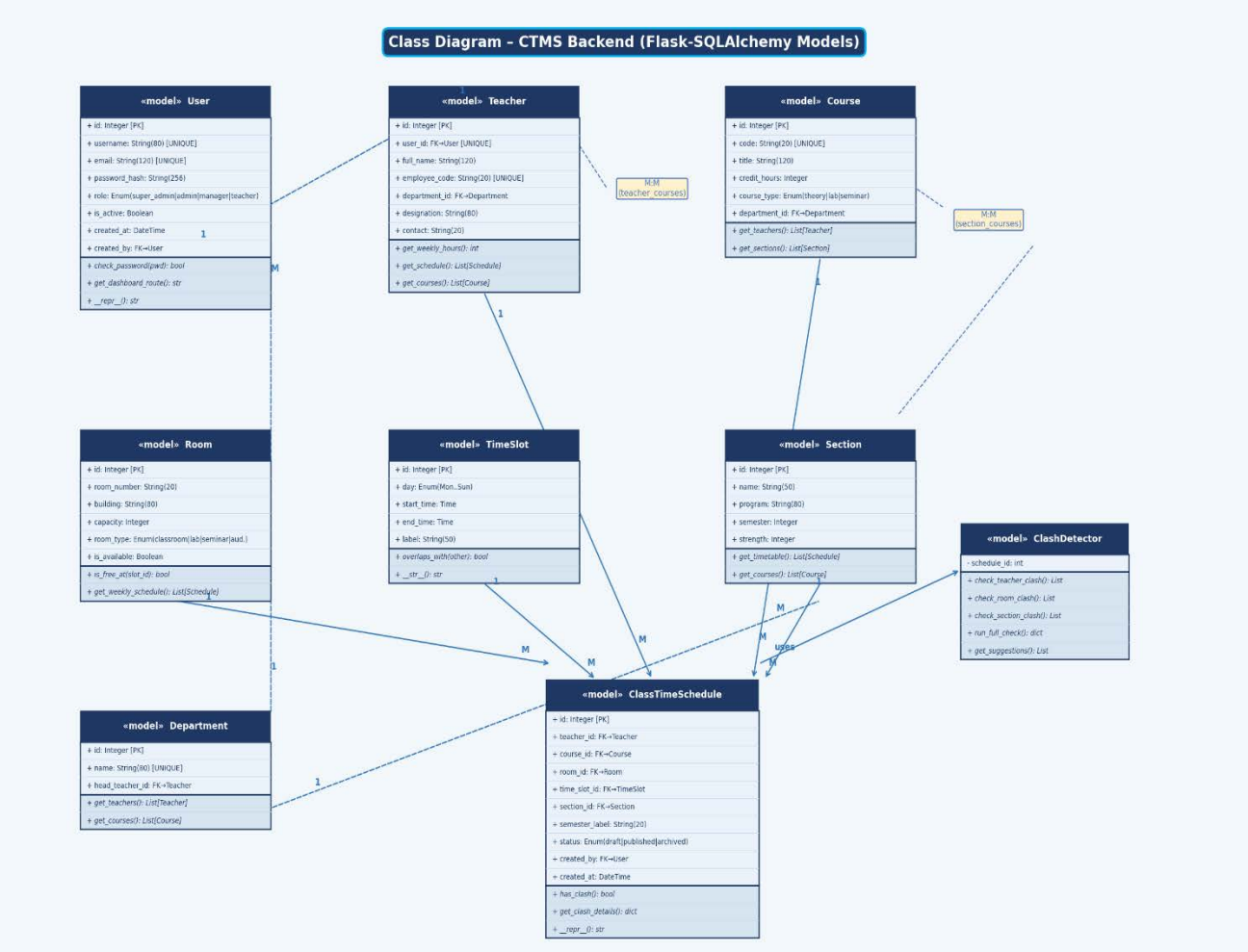

Activity Diagram – Teacher: View Personal Timetable
Teacher
System
Open Login Page
Enter Credentials
Validate Credentials?
Invalid
Valid
Show Error
'Invalid Credentials'
Create Session
Detect Teacher Role
Query schedule WHERE
teacher_id = current
Render Weekly
Timetable Grid
View Timetable
(Today highlighted)
Click a class for details?
Yes
No
Show Popup:
Course/Room/Section
Export PDF?
Yes
No
Generate &
Download PDF

5.5 Class Diagram (Backend Python/Flask Model Reference)

The class diagram directly corresponds to the SQLAlchemy model classes. Each model class maps to one database table. Relationships are expressed as Python attributes using SQLAlchemy relationship() declarations.

Class (Model)	Attributes / Fields	Methods & Relationships
User	id: int (PK) username: str email: str password_hash: str role: RoleEnum is_active: bool created_by: int (FK)	check_password(pwd) → bool set_password(pwd) → None can_manage(target_role) → bool to_dict() → dict created_users: List[User] (self-referential, one-to-many) teacher_profile: Teacher (one-to-one, optional) schedules_created: List[ClassTimeSchedule]
Teacher	id: int (PK) user_id: int (FK, UNIQUE) employee_code: str department: str designation: str phone: str max_hours_per_week: int	get_weekly_hours(semester, year) → int get_free_slots(day) → List[TimeSlot] has_clash_at(time_slot_id) → bool user: User (one-to-one) courses: List[Course] (many-to-many via CourseTeacher) schedules: List[ClassTimeSchedule] (one-to-many)
Course	id: int (PK) code: str (UNIQUE) title: str credit_hours: int course_type: CourseTypeEnum department: str is_active: bool	get_teachers() → List[Teacher] to_dict() → dict teachers: List[Teacher] (many-to-many via CourseTeacher) schedules: List[ClassTimeSchedule]
Room	id: int (PK) room_number: str (UNIQUE) building: str capacity: int room_type: RoomTypeEnum has_projector: bool is_available: bool	is_free_at(time_slot_id) → bool get_schedule(semester, year) → List[ClassTimeSchedule] schedules: List[ClassTimeSchedule] (one-to-many)
TimeSlot	id: int (PK) day_of_week: DayEnum start_time: time end_time: time slot_label: str is_break: bool	overlaps_with(other: TimeSlot) → bool duration_minutes() → int schedules: List[ClassTimeSchedule] (one-to-many)
Section	id: int (PK) name: str (UNIQUE) semester: int academic_year: str program: str student_count: int is_active: bool	get_timetable(year) → List[ClassTimeSchedule] copy_to_semester(new_sem, new_year) → int schedules: List[ClassTimeSchedule] (one-to-many)
CourseTeacher (Association)	id: int (PK) teacher_id: int (FK) course_id: int (FK) is_primary: bool	teacher: Teacher course: Course
ClassTimeSchedule	id: int (PK) section_id: int (FK) course_id: int (FK) teacher_id: int (FK) room_id: int (FK) time_slot_id: int (FK) semester: int	check_clashes() → ClashReport to_grid_dict() → dict to_export_row() → dict section: Section course: Course teacher: Teacher room:

	academic_year: str session_type: SessionTypeEnum created_by: int (FK) is_active: bool	Room time_slot: TimeSlot creator: User
ClashDetectionService (Service Class)	db_session: Session	check_teacher_clash(entry) → List[Clash] check_room_clash(entry) → List[Clash] check_section_clash(entry) → List[Clash] full_semester_report(semester, year) → ClashReport suggest_alternatives(clash) → List[Suggestion] uses: ClassTimeSchedule, Teacher, Room, Section, TimeSlot



5.6 Entity-Relationship (ER) Diagram Specification

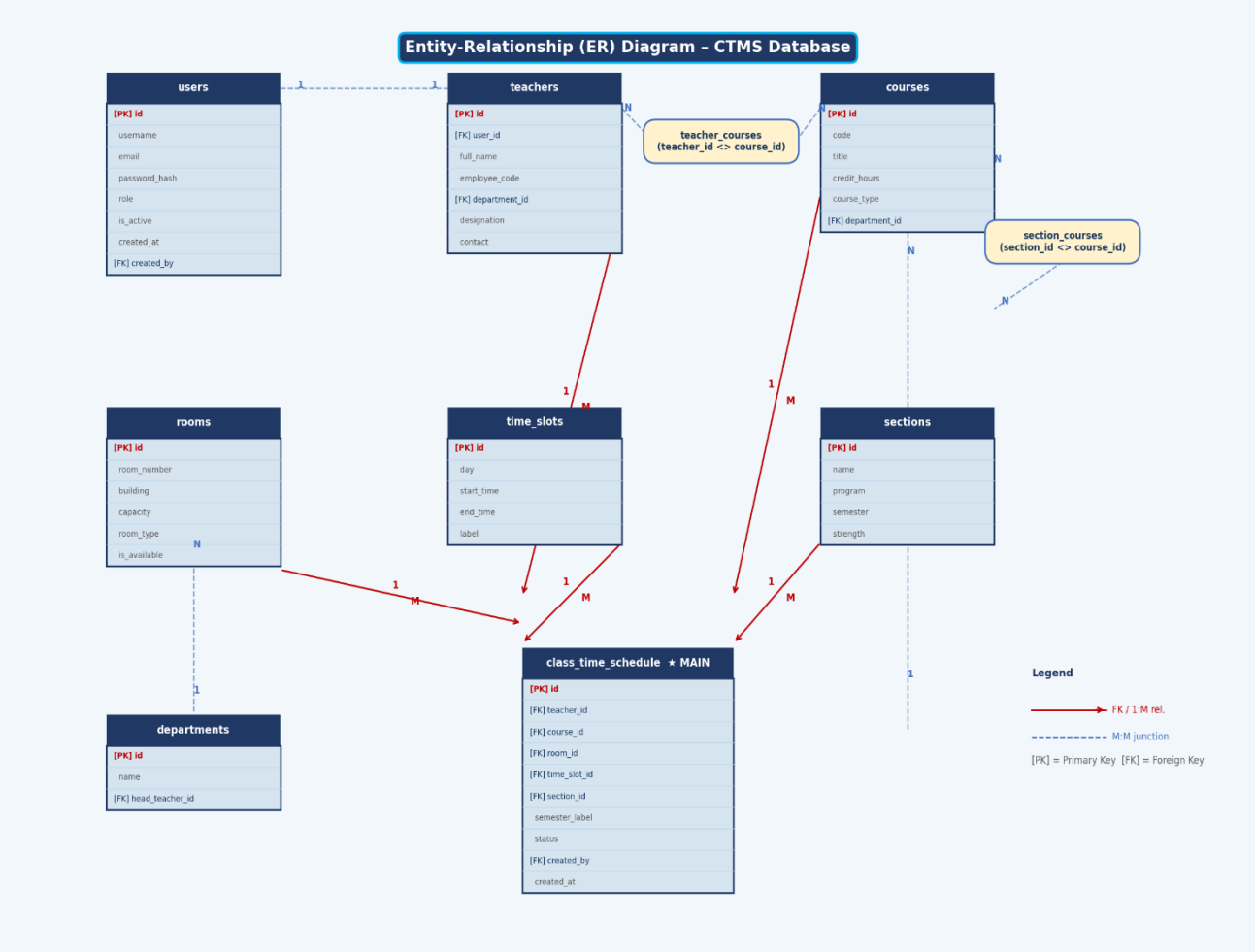
The following table specifies all entity relationships. Use this specification to draw the formal ER diagram in a diagramming tool. Relationship cardinality is expressed in crow's foot notation.

Relationship	Entity A	Cardinality	Entity B	FK Location	Notes
User creates User	users (creator)	1 : M	users (created)	users.created_by → users.id	Self-referential, nullable (Super Admin has NULL)
User has Teacher Profile	users	1 : 0..1	teachers	teachers.user_id → users.id UNIQUE	One-to-one; not every user has a teacher profile
Teacher teaches Courses	teachers	M : M	courses	course_teacher junction table	Many-to-many with is_primary attribute
Teacher has Schedules	teachers	1 : M	class_time_schedule	class_time_schedule.teacher_id	One teacher → many schedule entries
Course has Schedules	courses	1 : M	class_time_schedule	class_time_schedule.course_id	One course → many schedule entries
Room has Schedules	rooms	1 : M	class_time_schedule	class_time_schedule.room_id	One room → many time slots used
TimeSlot has Schedules	time_slots	1 : M	class_time_schedule	class_time_schedule.time_slot_id	One slot → many courses scheduled in it
Section has Schedules	sections	1 : M	class_time_schedule	class_time_schedule.section_id	One section → many weekly

					class entries
User creates Schedules	users	1 : M	class_time_schedule	class_time_schedule.created_by	Audit trail: who created entry

Key ER Diagram Notes:

- class_time_schedule is the central fact table with five foreign keys, making it the hub of the star schema.
- The course_teacher junction table implements the many-to-many relationship between teachers and courses with an additional is_primary boolean attribute.
- users has a self-referential one-to-many relationship (created_by) implementing the role hierarchy audit trail.
- teachers.user_id has a UNIQUE constraint enforcing the one-to-one relationship with users.
- All entity tables use INTEGER surrogate PKs for performance in JOIN-heavy clash detection queries.



6. External Interface Requirements

6.1 Login Page

- URL: /login (only public-facing page)
- Fields: Username or Email, Password (masked), Remember Me checkbox
- Button: Login → POST /login
- Behavior: On success → redirect to role-specific dashboard. On failure → flash error, lockout after 5 failures.
- No signup/register link is shown. Password reset link available.

6.2 Role-Specific Dashboards

- Super Admin Dashboard (/dashboard/superadmin): System health cards, user counts per role, clash count widget, recent activity log table, quick-action buttons.
- Admin Dashboard (/dashboard/admin): Current semester stats, entity counts, clash alerts, manage links.
- Manager Dashboard (/dashboard/manager): My sections, clash alert count, teacher load bar chart, Add Schedule shortcut.
- Teacher Dashboard (/dashboard/teacher): Weekly timetable grid (read-only), total hours, upcoming classes list, own clash warnings.

6.3 Timetable Grid View

- Layout: Rows = Time Slots, Columns = Days of Week (Mon–Fri/Sat). Cells show Course Code + Section + Room.
- Color coding: Normal entries (blue), Clashed entries (red), Break slots (gray), Empty slots (white).
- Controls: Filter by Section / Teacher / Room / Semester. Toggle between Grid View and List View.
- Clash cells are clickable to view clash detail popup.

6.4 CRUD Form Pages

- Each entity (teachers, courses, rooms, etc.) has a List page with search/filter and an Add/Edit form page.
- Forms use client-side validation (HTML5 + jQuery) and server-side validation (Flask-WTF).
- Flash messages confirm success or display field-level errors.
- Confirmation modal shown before delete/deactivate operations.

6.5 Clash Detection UI

- On Schedule Create/Edit form: Clash Check button triggers AJAX call to /api/check-clash returning JSON.
- Clash results shown inline below the form without page reload.
- Full Clash Report page: paginated table of all clashes, filterable by type/semester. Each row has a Fix button linking to the edit form.

6.6 Import / Export UI

- Export: Dropdown on List pages → CSV, Excel, PDF. Triggers file download.

- Import: Dedicated /import page with file upload dropzone, template download link, and preview table post-upload.
- Import preview highlights rows with errors in red and valid rows in green before final commit.

7. Non-Functional Requirements

NFR-ID	Category	Requirement
NFR-01	**Performance**	The schedule list page must load within 2 seconds for up to 500 active schedule entries.
NFR-02	**Performance**	Clash detection API endpoint (/api/check-clash) must respond within 500ms for a single entry check.
NFR-03	**Performance**	Full semester clash report must generate within 5 seconds for up to 1,000 schedule entries.
NFR-04	**Security**	All passwords stored as bcrypt hashes with cost factor ≥ 12.
NFR-05	**Security**	All routes protected by Flask-Login. Unauthorized access redirects to /login with 401 response.
NFR-06	**Security**	CSRF protection via Flask-WTF on all POST forms.
NFR-07	**Security**	SQL injection prevention enforced by SQLAlchemy ORM (no raw SQL for user inputs).
NFR-08	**Security**	Session cookie set with Secure, HttpOnly, SameSite=Strict flags.
NFR-09	**Reliability**	System must have ≤ 1 hour downtime per month in production.
NFR-10	**Reliability**	Database must support daily automatic backups. Recovery Point Objective (RPO) ≤ 24 hours.
NFR-11	**Usability**	All CRUD operations completable within 3 clicks from the dashboard.
NFR-12	**Usability**	Timetable grid must be readable on screens 1024px wide or wider (responsive Bootstrap layout).
NFR-13	**Usability**	Error messages must be specific (e.g., 'Teacher Dr. Ali has a clash with CS-301 at Monday 09:00'), not generic.
NFR-14	**Maintainability**	All Flask routes must be organized in Blueprints by module (auth, users, schedule, clash, etc.).
NFR-15	**Maintainability**	SQLAlchemy models separated in models/ package; templates in templates/ with base inheritance.
NFR-16	**Scalability**	Architecture must support switching from SQLite to PostgreSQL with no code changes (via DATABASE_URL env variable).
NFR-17	**Compatibility**	Application must run on Python 3.10+ on Linux/Ubuntu and Windows 10/11 environments.
NFR-18	**Auditability**	Every CREATE, UPDATE, DELETE action on any table must be logged in an activity_log table with user_id, action, table_name, record_id, and timestamp.